

ACERO — Arquitectura y flujo técnico (guía interna)

Para entender tu programa por dentro: qué módulo hace qué, cómo viajan los datos, y qué botón del dashboard dispara qué función. Todo referenciado a archivos reales del repo.

0. La regla que explica el diseño

La IA nunca es evidencia. El techo es la revisión humana. Nada es un descubrimiento.

Codex (el LLM) **propone, razona y redacta**; la **evidencia** sale de **código que corre sobre datos reales** en una caja aislada. Todo el sistema está construido para *impedirse a sí mismo* fabricar positivos. Si entiendes esto, entiendes por qué hay tantos controles.

1. Arquitectura en una imagen



Es un **monolito modular** de Python 3.12. No hay microservicios: todo corre en un proceso, con hilos para el paralelismo. La UI es una SPA sin build (archivos estáticos servidos tal cual, con CSP estricta).

2. Persistencia — dónde vive cada cosa

Capa	Archivo	Qué guarda
------	---------	------------

Ledger	<code>src/acero/ledger/ (service.py, models.py)</code>	proyectos + procedencia (cada evento: quién/qué/cuándo creó o cambió algo)
Discovery store	<code>src/acero/discovery/store.py</code>	los OBJETOS de la investigación, por kind
World Model	<code>src/acero/world_model/graph.py</code>	grafo de conocimiento (nodos = claims con confianza)
Vault	Obsidian (<code>obsidian_sync.py</code>)	espejo Markdown indexable de TODO (el "cerebro")

Los **kind** del discovery store (la taxonomía central): **brief** (tema/pregunta libre) · **candidate** (hipótesis) · **literature** (papers con DOI) · **experiment** · **negative** (resultado negativo, **nunca se borra**) · **dossier** · **critique** (objeciones del Revisor) · **mission** · **synthesis**.

Guardarraíl del store (`store.delete`): negativos y candidatos REJECTED **no se borran** salvo `force=True` — que sólo usa el borrado explícito del humano, y sólo *después* de archivar la memoria en el vault (`lifecycle.delete_hypothesis`).

3. El flujo completo, etapa por etapa

Etapa 1 — Tema → proyecto

- **Botón:** ■ **Nuevo proyecto** → POST `/api/workspace/project`
- **Código:** `workspace.create_project(title, domain, topic)` guarda el `topic` como un objeto `kind="brief"`. Ese texto libre es la **semilla** de todo.

Etapa 2 — El programa genera hipótesis (Codex)

- **Botón:** ■ **Generar hipótesis** → POST `/api/projects/{id}/hypotheses/generate`
- **Código:** `hypotheses.generate()` lee el **brief**, inyecta el tema en el prompt y pide a Codex hipótesis **rivales** (principal + nula + alternativas), cada una con: pregunta detonante, argumento, duda (qué la falsaría), con-qué-compite, cómo probarla, y **kind** (novel/theorized/open_question). Se guardan como **candidate** en estado **PROPOSED**.

Etapa 3 — Análisis epistémico (EVA + preguntas) ← *parte que reforzamos*

- **Botón:** ■ **Preguntas (EVA)** → POST `/api/projects/{id}/epistemic/questions`
- **Código:** `epistemic_bridge.run_epistemic(pid, extractor=make_codex_extractor())`
- Cada hipótesis se **reconstruye** en un **ClaimRecord** (`epistemic/claim_reconstructor.py`). El extractor Codex (`codex_extract`) puebla exposición, outcome, mecanismo, supuestos y condiciones de frontera **por hipótesis**; si Codex no está, cae al **heuristic_extract** (marcando `provenance = llm/heuristic/fallback` y una **confidence**).
- **EVA** (`epistemic/eva.py::audit_external` → `vulnerability.scan_vulnerabilities`) genera la superficie de **vulnerabilidades** (fuente única, no replicado, **confusión**, causalidad inversa, extrapolación, supuestos no validados, mecanismo ambiguo). Cada una trae su prueba decisiva y su sonda más barata.
- El **motor de preguntas** (`questions/question_engine.py`) convierte cada vulnerabilidad en una **pregunta investigable**, priorizada por 15 componentes (claridad, falsabilidad, poder discriminante, disponibilidad de datos, coste...).
- La **prueba discriminante** (`epistemic/discrimination` vía pipeline) mide en **bits** cuánto separaría a las rivales.

- **De-duplicación semántica:** marca claims con vulnerabilidades de **contenido** idéntico (no sólo tipo) — para no repetir crítica.

Etapa 4 — Aprobación (HUMANO)

- **Botón:** ✓ **Aprobar** en la tarjeta → `POST /hypothesis/{id}/status` con razón obligatoria.
- Sólo las hipótesis **APPROVED** pueden correr misiones. Rechazar (**REJECTED**) las saca del tablero activo **con todo lo relacionado** (`phases.py` filtra) y las manda a un cajón recuperable.

Etapa 5 — Misión: el ciclo autónomo (Codex + datos reales)

- **Botón:** ■ **Lanzar misión** → `POST /hypothesis/{id}/mission`
- **Código:** `missions.MissionEngine.start()`. Ver sección 4.

Etapa 6 — Dossier (borrador) + techo humano

- La síntesis produce un **dossier** en estado **DRAFT / requiere revisión humana**. El sistema **se detiene aquí a propósito**. Cruzar a "descubrimiento" es decisión humana.

4. El Mission Engine en detalle (`src/acero/portal/missions.py`)

Una misión corre el pipeline completo de UNA hipótesis, sin clics intermedios:

```
STEPS = investigate → experiments_propose → experiments_run → synthesize → rigor_loop
        (peso 12)      (12)                  (46, el más largo) (10)      (20)
```

Cada paso (`_run_step`) delega en `hypothesis_flow.py`: - **investigate:** `fl.investigate()` — busca **literatura real** (multi-fuente, DOI + chequeo de retracción) y confronta la hipótesis con lo publicado. - **experiments_propose:** Codex **diseña** experimentos (qué, cómo, controles, discriminador, fuente de datos). - **experiments_run:** por cada experimento, `experiment_factory.run_generated()` (sección 5). - **synthesize:** `synthesis.synthesize_hypothesis()` — vuelca resultados al World Model y arma el dossier. - **rigor_loop:** el **Revisor (Aristóteles)** critica, convierte sus objeciones en nuevos experimentos, los corre y re-sintetiza (se auto-machaca).

Robustez (persistencia): - Cada paso se **checkpointea** en el ledger ANTES y DESPUÉS (`kind="mission"`). Un reinicio no pierde trabajo: `resume_pending()` retoma **desde el siguiente paso**. - **Concurrencia:** `MAX_MISSIONS` (env `ACERO_MAX_MISSIONS`, default 4) misiones a la vez, vía `ThreadPoolExecutor`. Cada una lanza su propio `codex exec` **efímero** (no hay sesión compartida → paralelizan de verdad). - **Watchdog auto-sanador** (`_stale_action + watchdog()`), llamado throttled desde `list_missions`: una misión cuyo worker murió se **re-lanza** (resume desde checkpoint); una colgada >20 min se marca **FAILED** (reintentable). El dashboard se cura solo, sin reiniciar.

5. La Fábrica de Experimentos (`src/acero/portal/experiment_factory.py`)

Es donde "la IA nunca es evidencia" se hace código. Flujo de `run_generated()`:

```
1. PLAN      default_plan() → Codex elige qué datasets publicos hacen falta (o ninguno).
2. FETCH     fetch data()   → descarga real, sólo de hosts permitidos (allowlist).
3. SCHEMA    _schema()     → introspección REAL de columnas + n° de filas (codegen no es ciego).
4. CODEGEN   build_codegen_prompt() → Codex escribe UN script Python (stdlib+numpy).
5. SANDBOX   se corre AISLADO: sin red, sin subprocess, sin leer fuera de ./, determinista.
6. REPAIR    si falla, reintentado con el error como feedback (bucle acotado).
7. CROSS-CHECK _SECOND_IMPL → una SEGUNDA implementación con enfoque distinto; compare results
```

```
compara. Si discrepan, ACERO DEGRADA el veredicto (no cuenta un 'supports').
8. VERDICT se decide por el DISCRIMINADOR contra los NULOS. Contrato de salida:
RESULT_JSON: {metrics, null_test, verdict: supports|refutes|inconclusive,
              verdict_reason, anomalies}
```

Reglas anti-autoengaño incrustadas en el prompt (`_CODE_RULES`, `join_hint`): - Prueba nula obligatoria (permutación / surrogatos / Monte Carlo). - "supports" exige **segunda implementación independiente** que concuerde. - **Integración de datos:** preferir columnas **in-table**; si hay que unir, por **ID estable** (no por nombre); **guardarraíl de cobertura** (si la retención cae por debajo del umbral → **inconclusive** marcando "defecto de cross-match, no ausencia de señal"). - Nunca inventar números: toda métrica sale del cómputo.

6. La Constitución Científica (`src/acero/science/`)

Es la capa que separa "candidato honesto" de "positivo fabricado". Módulos clave:

- **Máquina de estados** (`states.py`): escalera de 0 a 12. **Techo de ACERO = 9 (CANDIDATO_A_PREPRINT)**. Los estados 10-12 (revisión por pares, publicado, replicado externamente) son **sólo humanos/externos**. No se puede saltar de exploratorio a confirmatorio sin pasar por "exploratorio robusto".
- **Pre-registro** (`preregistration.py`): congela hipótesis+variable+prueba+regla de decisión y su **hash** ANTES de abrir datos nuevos. Distingue régimen de **descubrimiento** (explora libre) vs **confirmación** (plan congelado).
- **Holdout** (`holdout.py`): mantiene datos confirmatorios **inaccesibles** hasta congelar.
- **Independencia** (`independence_graph.py`): calcula el **nivel** (0=mismo dato ... 8=externo total). Un split del mismo dataset **jamás** cuenta como replicación. Dos fuentes con la misma **raíz de curación** no son independientes aunque parezcan distintas.
- **Claim Compiler** (`claim_compiler.py`): traduce la evidencia a la **frase máxima permitida** ("asociado con" / "predice en esta población" / "efecto bajo estos supuestos" / "replicado") y **lintea sobreafirmaciones** ("demuestra", "causa" sin respaldo).
- **Panel adversarial** (`panel.py`): 8 personas con mandatos incompatibles (estadístico, causalista, detective de datos, revisor de novedad...). Preservan el desacuerdo; si un crítico "duro" bloquea, se detiene.
- **Nulos por estructura** (`nulls.py`), **presupuesto de incertidumbre** (`uncertainty_budget.py`), **CAUSA** (DAG + backdoor por d-separación, `causal.py`).

7. El proveedor LLM (`src/acero/llm/providers.py`)

- `CodexCliProvider` shelea `codex exec --json --ephemeral -s read-only`. Cada llamada es una **sesión efímera aislada** en su propio tempdir → múltiples corren en paralelo.
- `complete()` para texto; `complete_json(prompt, schema)` para salida estructurada vía `--output-schema`. **Ojo:** ese modo es **estricto** — el schema debe listar *todas* las propiedades en `required` (como `PLAN_SCHEMA` y `EXTRACT_SCHEMA`).
- Se activa con `ACERO_LLM_PROVIDER=codex`. Consume la cuota del usuario; por eso no es el default silencioso.

8. Mapa botón-del-dashboard → endpoint → función

Botón / acción	Endpoint	Función
■ Nuevo proyecto	<code>POST /api/workspace/project</code>	<code>workspace.create_project</code>

■ Generar hipótesis	POST /projects/{id}/hypotheses/generate	hypotheses.generate
■ Preguntas (EVA)	POST /projects/{id}/epistemic/questions	epistemic_bridge.run_epistemic
✓ Aprobar / Rechazar	POST /hypothesis/{id}/status	hypothesis_flow.set_status
■ Borrar (cascada)	POST /hypothesis/{id}/delete	lifecycle.delete_hypothesis
■ Lanzar misión	POST /hypothesis/{id}/mission	missions.MissionEngine.start
■ Investigar literatura	POST /hypothesis/{id}/investigate	hypothesis_flow.investigate
(fases del dashboard)	GET /projects/{id}/phases	phases.build_phases
(lista de misiones)	GET /projects/{id}/missions	MissionEngine.list_missions (+watchdog)
Procesos activos	GET /api/processes	lifecycle.active_processes

`phases.build_phases()` es el que arma TODO el tablero: filtra REJECTED, calcula señales por hipótesis (literatura/experimentos/dossier), KPIs, mini-reportes por fase y el cajón de rechazadas.

9. Qué mejoramos en esta iteración (y por qué importa)

1. **EVA específico por hipótesis** — antes daba una crítica plantillada idéntica a todas; ahora reconstruye cada hipótesis (extractor Codex) y **detecta la confusión** que era el eje real del caso. Benchmark: cobertura de confusión 0→100%, supuestos 3→7 por claim.
2. **Codegen consciente de esquema** — antes un `hint` premiaba unir catálogos **por nombre**, y eso convirtió una muestra de 4083 planetas en 3. Ahora prefiere columnas in-table, exige ID estable y aborta si la cobertura del cruce es insuficiente.
3. **Watchdog de misiones** — antes una misión cuyo worker moría quedaba "90% para siempre". Ahora el dashboard la retoma o la marca fallida sola.

Todo con pruebas offline, ruff y mypy limpios, en commits pequeños.

10. Cómo leer un resultado sin engañarte

- **verdict = inconclusive** casi siempre es lo honesto: significa que los datos no alcanzan, o que un control (placebo/nulo/cross-check) tumbó la señal. **No es un bug**.
- **"degradado por ACERO"** = la segunda implementación discrepó → no se cuenta. Es el sistema protegiéndote.
- **Nivel de independencia 1** = una sola raíz de datos → **no** es replicación.
- El **dossier DRAFT** es un borrador para TI. El techo eres tú.

Una línea: ACERO hace el trabajo pesado y reproducible de investigar, y se detiene, a propósito, justo antes de la palabra "descubrimiento".